AI – TECH

SKRYPT DO LABORATORIUM

PRZETWARZANIE MULTIMEDIÓW W SYSTEMACH DECYZYJNYCH

LABORATORIUM 3:

Przegląd algorytmów uczących się metryk dystansu dla danych wizyjnych

Adam Kurowski

Projekt współfinansowany ze środków Unii Europejskiej w ramach Europejskiego Funduszu Rozwoju Regionalnego
Program Operacyjny Polska Cyfrowa na lata 2014-2020,
Oś Priorytetowa nr 3 "Cyfrowe kompetencje społeczeństwa" Działanie nr 3.2 "Innowacyjne rozwiązania na rzecz aktywizacji cyfrowej"
Tytuł projektu: „Akademia Innowacyjnych Zastosowań Technologii Cyfrowych (AI Tech)”

1. Zagadnienia wstępne

1.1.Wymagania w odniesieniu do studenta

W ramach przygotowania do zajęć, student powinien powtórzyć lub zapoznać się z podstawowymi zagadnieniami związanymi z analizą reprezentacji, metrykami dystansu i ogólną wiedzą dotyczącą programowania. Przed przystąpieniem do wykonania ćwiczenia zaleca się:

- powtórzenie wiedzy nabytej w czasie wykładów,
- zapoznanie się (przed ćwiczeniem) z instrukcją do ćwiczenia ilustrującą podstawowe zagadnienia z zakresu tematyki ćwiczenia.

Ponadto wymagana jest od studenta:

- podstawowa umiejętność programowania,
- podstawowa znajomość języka Python,
- podstawowa znajomość algorytmów uczenia maszynowego przedstawionych w trakcie trwania wykładów
- umiejętność rozwiązywania problemów.

Wyżej wymienione wymagania bazują na wiedzy i umiejętności jakie studenci powinni zdobyć w czasie edukacji na I poziomie studiów oraz w trakcie trwania 2. semestru studiów II stopnia. W przeciwnym przypadku studenci powinni nadrobić samodzielnie stosowny materiał.

1.2.Wymagania w odniesieniu do stanowiska laboratoryjnego

Stanowisko laboratoryjne powinno być wyposażone w komputer z dostępem do sieci komputerowej oraz z następującymi zasobami:

- przykładowe programy (kody programów) podane w załączniku
- zainstalowany język Python w wersji przynajmniej 3.6.5
- Jupyter Notebook
- biblioteki niezbędne do realizacji ćwiczenia
- przeglądarka WWW.

Wykaz bibliotek języka Python wraz z numerami wersji i odnośnikami URL do ich dokumentacji w formie on-line zamieszczony jest poniżej:

Nazwa biblioteki	**Wersja**	**Odnośnik do dokumentacji**
Matplotlib	1.18.1	https://matplotlib.org/
NumPy	3.0.3	https://numpy.org/doc/stable/reference/
Pandas	0.24.1	https://pandas.pydata.org/
SciPy	1.5.4	https://docs.scipy.org/doc/scipy/reference/
Scikit-learn	6.1.0	https://scikit-learn.org/stable/

Przed przystąpieniem do wykonywania ćwiczenia warto zapoznać się z ogólnymi opisami narzędzi dostępnymi we wspomnianych bibliotekach, gdyż często znajduje się tam wiele gotowych, przydatnych i co najważniejsze – szeroko używanych, przetestowanych narzędzi realizujących różne skomplikowane zadania. Pozwala to na uniknięcie implementowania od początku algorytmów, które już wcześniej zostały przez kogoś zaprogramowane w dobry, szeroko zweryfikowany przez użytkowników biblioteki sposób.

Istnieje możliwość realizacji ćwiczenia w zewnętrznym środowisku Google Colaboratory na platformie Google, gdzie wymagane jest od użytkownika posiadanie konta Google, natomiast nie ma potrzeby konfiguracji środowiska.

1.3.Cele ćwiczenia

Celem ćwiczenia jest praktyczna demonstracja wiedzy teoretycznej, przedstawionej w ramach wykładów. Realizując ćwiczenie laboratoryjne studenci zdobędą szereg nowych umiejętności w zakresie poprawnego doboru i używania algorytmów uczących się metryk dystansu. Zademonstrowane zostanie także praktyczne wykorzystanie tego typu algorytmów do wyszukiwania i klasyfikowania wzorców.

1.4. Spodziewane efekty kształcenia - umiejętności i kompetencje:

Studenci w ramach zajęć nabędą wiedzę dotyczącą między innymi następujących zadań:

- przygotowania danych do analizy algorytmami uczącymi się metryk dystansu
- przeprowadzać trening sieci neuronowych wykorzystując metody contrastive loss i triplet loss
- analizować jakość reprezentacji wytwarzanych przez sieci trenowane wyżej wymienionymi metodami,
- wykorzystywać wyuczone metryki dystansu oraz sposoby generowania reprezentacji wzorców do ich wyszukiwania i klasyfikacji.

1.5.Metody dydaktyczne

Zadanie rozpocznie się od konfiguracji środowiska programistycznego, którym może być zarówno środowisko on-line takie jak usługa Google Colaboratory, jak i lokalnie uruchamiane oprogramowanie Jupyter. Dalsze działania odbywać się będą na podstawie dodatkowych materiałów i demonstracji prowadzącego, studenci będą wykonywać zadania przygotowane w postaci interaktywnych dokumentów Jupytera. Konieczne jest zarówno przyswojenie treści przekazywanych przez prowadzącego, jak i znajomość wstępnych, podstawowych zagadnień niezbędnych do zrozumienia omawianych tematów. Na początku przedstawiane są proste problemy i demonstracje, kolejne zadania

są coraz bardziej skomplikowane, przedstawiają coraz bardziej zaawansowane pojęcia, oraz wymagają bardziej intensywnego i kreatywnego zaangażowania ze strony wykonującego ćwiczenie studenta.

Wszystkie kody źródłowe programów wpisuje w dokumencie elektronicznym (lub pokazuje prowadzącemu), który jednocześnie staje się sprawozdaniem z ćwiczenia laboratoryjnego.

1.5.Zasady oceniania

Ocenie podlegać będzie realizacja poszczególnych zadań w ramach danego ćwiczenia laboratoryjnego. W trakcie realizacji ćwiczenia możliwe jest przydzielenie przez prowadzącego dodatkowych zadań oraz zadawanie pytań dotyczących ćwiczenia. Premiowane są także wszelkie próby wyciągnięcia i uogólnienia wniosków wypływających z wykonywanych ćwiczeń. Mogą one dotyczyć zarówno aspektów teoretycznych, jak i praktycznych wykonywanych analiz.

Poszczególnym zadaniom przydzielono liczbę punktów jakie można zdobyć za ich wykonania. Maksymalna liczba punktów wynosi 10.

Wykaz literatury podstawowej do ćwiczenia:

Wykaz literatury podstawowej:

1.	Skrypt z materiałami do przedmiotu „Przetwarzanie multimediów w systemach dezycyjnych”
2.	Materiały do przedmiotu opracowane w formie edukacji na odległość, dostęp: https://enauczanie.pg.edu.pl/moodle/course/view.php?id=12525
3.	Aurélien Géron, Uczenie maszynowe z użyciem Scikit-Learn i TensorFlow. Wydanie II, Helion 2020
4.	Sebastian Raschka, Python. Uczenie maszynowe, Helion 2017
5.	Aggarwal C., Hinneburg A., Keim A., On the Surprising Behavior of Distance Metrics in High Dimensional Space. In: Van den Bussche J., Vianu V. (eds) Database Theory — ICDT 2001. ICDT 2001. Lecture Notes in Computer Science, vol 1973. Springer, Berlin, Heidelberg 2001. https://doi.org/10.1007/3-540-44503-X_27
6.	Wang, F., Liu, H. Understanding the behaviour of contrastive loss. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (pp. 2495-2504). 2021
7.	Hermans, A., Beyer, L., & Leibe, B. (2017). In defense of the triplet loss for person re-identification. arXiv preprint arXiv:1703.07737.

Wykaz literatury uzupełniającej:

1.	Dokumentacja bibliotek wymienionych w sekcji dotyczącej wymagań względem stanowiska
2.	Dokumentacja platformy Colaboratory: https://colab.research.google.com/notebooks/intro.ipynb#scrollTo=-gE-Ez1qtyIA
3.	Francois Chollet, Deep Learning. Praca z językiem Python i biblioteką Keras, Helion 2019

2. Przebieg ćwiczenia

L.p.	Zadanie
1.	Zapoznanie się z instrukcją laboratoryjną (przed ćwiczeniem)
2.	Wprowadzenie kierownika ćwiczenia. Zapoznanie się uczestników ze stosowanymi programami, przegląd dokumentacji (10 min.)
3.	Badanie algorytmu contrastive loss (30 min.)
4.	Badanie algorytmu triplet loss (30 min.)
5.	Opracowanie klasyfikatorów progowych bazujących na badanych wcześniej modelach (30 min.)
6.	Wykorzystanie wybranego algorytmu do zrealizowania poszukiwania wzorca podobnego do przedstawionego przykładu (20 min.)
7.	Przesłanie/pokazanie zbioru dokumentów/programów do oceny (10 min.)

UWAGI

Przed przystąpieniem do ćwiczenia należy utworzyć własny, unikalny katalog w lokalizacji wskazanej przez prowadzącego, a następnie należy tam umieścić pliki testowe oraz własne implementacje kodu programów dokonujących rozwiązanie zadanie. Należy pracować wyłącznie na wykonanej kopii plików.

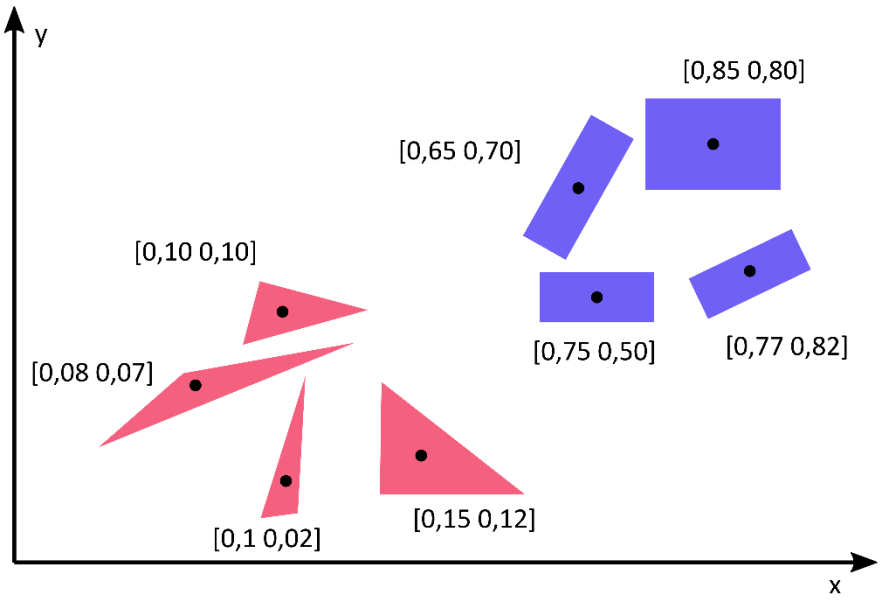
3. Wprowadzenie do ćwiczenia

Uczenie maszynowe wykorzystujące paradygmat treningu metryk dystansu jest techniką bardzo popularną w kontekście szeregu specyficznych zastosowań związanych z analizą podobieństwa wzorców. Wzorcami takimi mogą być zdjęcia, rysunki, nagrania wideo, czy też nagrania dźwiękowe. Przykładami tego typu analiz są:

- **porównywanie ze sobą złożonych wzorców** danych, np. w kontekście uwierzytelniania biometrycznego za pomocą głosu,
- **projektowanie klasyfikatorów**, które potrafią **rozróżniać znaczną liczbę klas** obiektów i dla których **liczba klas może się zmieniać**,

- projektowanie **algorytmów wyszukiwania treści multimedialnych podobnych do przykładów** podanych przez osobę korzystająca z algorytmu.

Zadania te można sprowadzić do wspólnego problemu polegającego na powiązaniu wzorców danych (np. zdjęć, ramek zapisu dźwięku, klatek filmu, itd.) z ich skróconym algebraicznym opisem. Opis taki ma postać wektora liczb (np. 128 wartości zmiennoprzecinkowych) i może być traktowany jak punkt w przestrzeni posiadających tyle samo wymiarów, ile składowych ma dany wektor. Wektor przypisany do wzorca multimedialnego w dalszej części instrukcji będziemy nazywać **wektorem reprezentacji**. Przestrzeń, w której znajdują się punkty odpowiadające wektorom reprezentacji nazywać będziemy **przestrzenią reprezentacji**. Przykład takiej przestrzeni, reprezentacji i powiązanych z tymi reprezentacjami wzorców można zobaczyć na rys. 1.



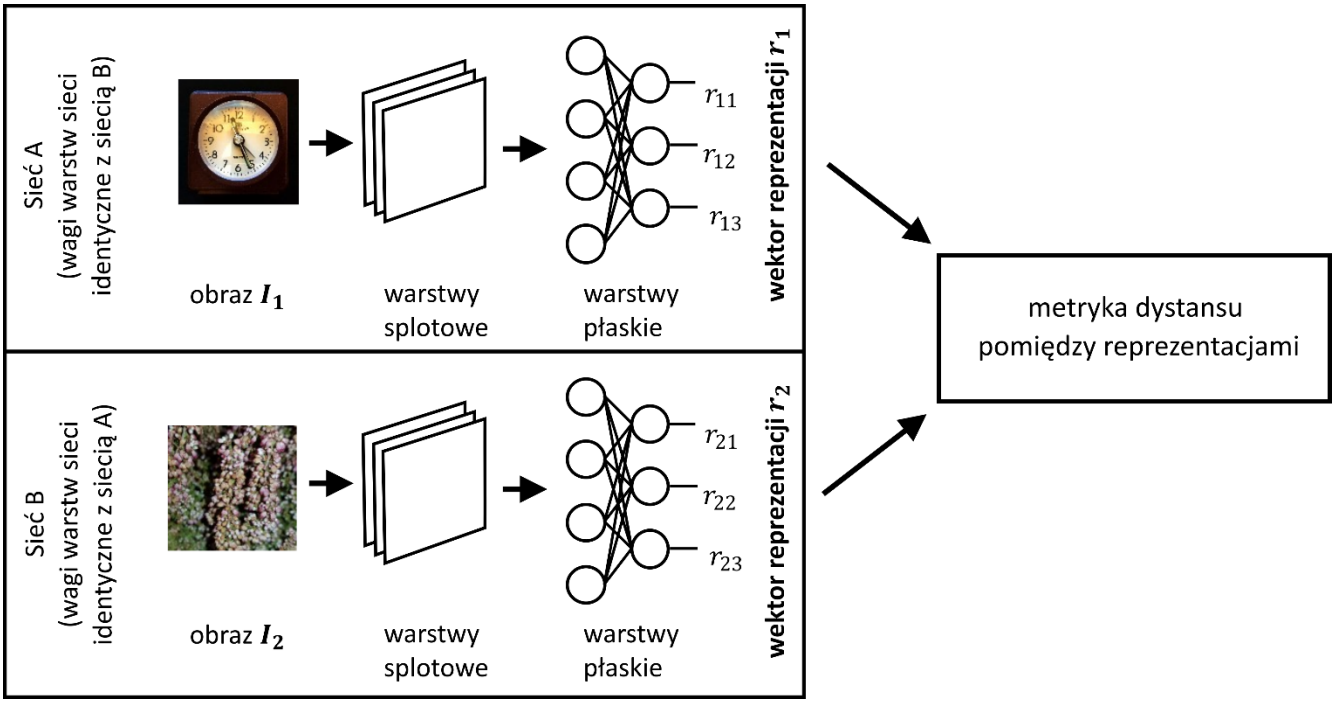
Rysunek T1. Przykładowe przypisanie kształtów do odpowiadających im wektorów reprezentacji. Osie x i y tworzą dwuwymiarową przestrzeń reprezentacji, w której umieszczone są punkty (dwuelementowe wektory wartości) powiązane z przypisanymi im obiektami (trójkąty i prostokąty).

Taki sposób przedstawienia opisu multimediów znacznie ułatwia porównywanie ich między sobą. Możliwe jest przeprowadzenie treningu modeli uczenia maszynowego w taki sposób, że punkty powiązane z danymi wzorcami multimedialnymi będą ułożone w przestrzeni decyzyjnej w taki sposób, aby odległości między nimi były proporcjonalne do miary podobieństwa lub miary braku podobieństwa (dystansu) pomiędzy dwoma wzorcami.

Algorytm uczący się metryk odległości składa się z kilku elementów:

- **dwóch sieci neuronowych dzielących między sobą wagi** – sieci te dokonują przekształcenia wzorca na ich wejściu na wektor reprezentacji. Sieć syjamska przyjmuje dwa porównywane ze sobą wzorce i przekształca je na dwie odpowiadające im reprezentacje,
- modułu przyjmujący **parę wektorów reprezentacji** z sieci syjamskiej, który **oblicza na ich podstawie metrykę odległości**.

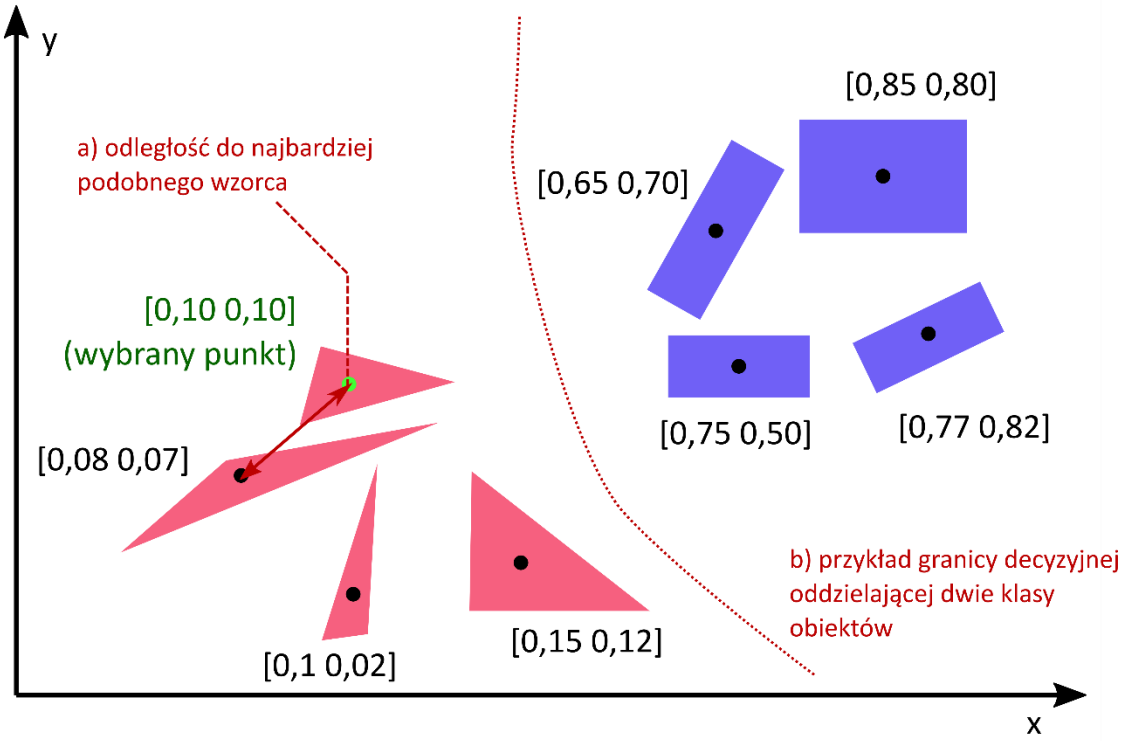
W teorii sieć syjamska ma strukturę taką jak na rys. 2. Sieci A i B są identyczne pod względem wykorzystywanych przez nie wag – zmiany w wagach jednej sieci oznaczają zmiany w wagach drugiej z nich. W praktyce zwykle w kodzie nie realizuje się wprost struktury sieci syjamskiej. Zwykle tworzy się tylko strukturę jednej z jej odnóg, natomiast zasada działania sieci realizowana jest poprzez wykorzystanie pojedynczego, zdefiniowanego pionu sieci dwukrotnie. Moduł obliczający odległość pomiędzy punktami często wchodzi w skład funkcji obliczającej funkcję straty. Czasami przez analogię do innych algorytmów nazywa się taką pojedynczą sieć realizującą funkcjonalność obu części sieci syjamskiej nazywa się **koderem**. Poprzez analizę geometrycznego rozmieszczenia punktów w przestrzeni reprezentacji możliwe jest wyznaczanie wzorców podobnych do siebie – **im dystans pomiędzy punktami mniejszy, tym wzorce te uznać możemy za bardziej do siebie podobne**. W taki właśnie sposób możliwe jest wykorzystanie algorytmów uczących się metryk odległości do wyszukiwania np. zdjęć wizualnie podobnych do podanego przykładu.



Rysunek 2. Ogólna struktura sieci syjamskiej – w praktyce bardzo często sieć A oraz sieć B to ta same jedna sieć uruchamiana w procesie treningu i określania dystansu dwukrotnie.

Można też potraktować przestrzeń reprezentacji jak przestrzeń decyzyjną dla klasyfikatora wykorzystującego do swojego działania odległość między reprezentacjami przykładów. Jeśli posłużymy się np. przykładem klasyfikatora kNN (k nearest neighbours, k najbliższych sąsiadów), to uzyskamy klasyfikator, które będzie miał tę właściwość, że nie będzie wymagał dodatkowego procesu treningu, jeśli postanowimy dodać nową klasę, którą ma on rozpoznawać. Wystarczy tylko do przestrzeni decyzyjnej/reprezentacji dodać nowe wzorce wytworzone przez koder i od razu klasyfikator ten zacznie identyfikować wzorce podobne do nowego wzorca z klasą tego wzorca. Na podobnej zasadzie możliwe jest zrealizowanie algorytmu uwierzytelniania biometrycznego, gdyż w istocie moduł uwierzytelniający

jest właśnie klasyfikatorem takiego typu, jak opisany. Przykłady obydwu opisanych zastosowań zwizualizowane są na rys. 3.



Rysunek 3. Przykład dalszego wykorzystania wektorów reprezentacji wytworzonych przez algorytmy uczące się metryk dystansu – określanie wzorca

W trakcie laboratorium badane będą dwa algorytmy uczące się metryk dystansu. Pierwszym, prostszym z nich będzie algorytm straty kontrastowej (ang. contrasitve loss). W algorytmie tym dwa wzorce (w naszym przypadku będą nim obrazy cyfr ze zbioru MNIST) podawane będą na wejścia sieci syjamskiej. Sieć ta następnie obliczy metrykę dystansu pomiędzy tymi wzorcami. Na wejścia sieci podawane będą zarówno pary przykładów należące do tej samej klasy (przedstawiające tę samą cyfrę), jak i pary

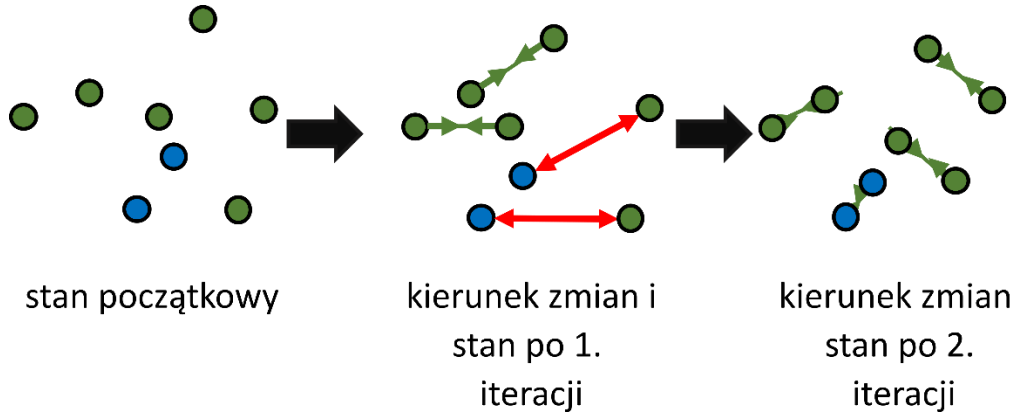
pochodzące z różnych klas (przedstawiające dwie różne cyfry). Funkcja straty obliczana jest według następującego wzoru

$$L_{\text{contrastive}} = \begin{cases} d(r_a, r_p) \text{ dla pary z tej samej klasy} \\ \max(0, m - d(r_a, r_n)) \text{ dla pary przykładów z różnych klas} \end{cases}, \quad (1)$$

gdzie:

- $L_{\text{contrastive}}$ to wartość straty kontrastowej,
- $d(r_a, r_p)$ to dystans pomiędzy wektorem reprezentacji r_a i r_p, gdzie wektor r_p pochodzi z tej samej klasy co r_a,
- $d(r_a, r_n)$ to dystans pomiędzy wektorem reprezentacji r_a i r_n, gdzie wektor r_n pochodzi z innej klasy
  niż r_a,
- m jest docelowym marginesem, który powinien oddzielać od siebie przykłady z różnych klas w przestrzeni reprezentacji.

Trenowanie sieci neuronowej na przykładach ze etykietowanego zbioru danych za pomocą straty kontrastowej wizualnie można sobie wyobrażać jak przybliżanie do siebie w kolejnych iteracjach treningu punktów pochodzących z tej samej klasy i oddalanie od siebie punktów pochodzących z różnych klas. Przykład tego procesu przedstawiony jest na rys. 4.



Rysunek 4. Przykład działania algorytmu contrastive loss. Punkty należą do dwóch klas (niebieskiej i zielonej). W kolejnych iteracjach punkty z tych samych klas są do siebie przybliżane (zielone strzałki), a punkty z różniących się klas są od siebie oddalane (czerwone strzałki).

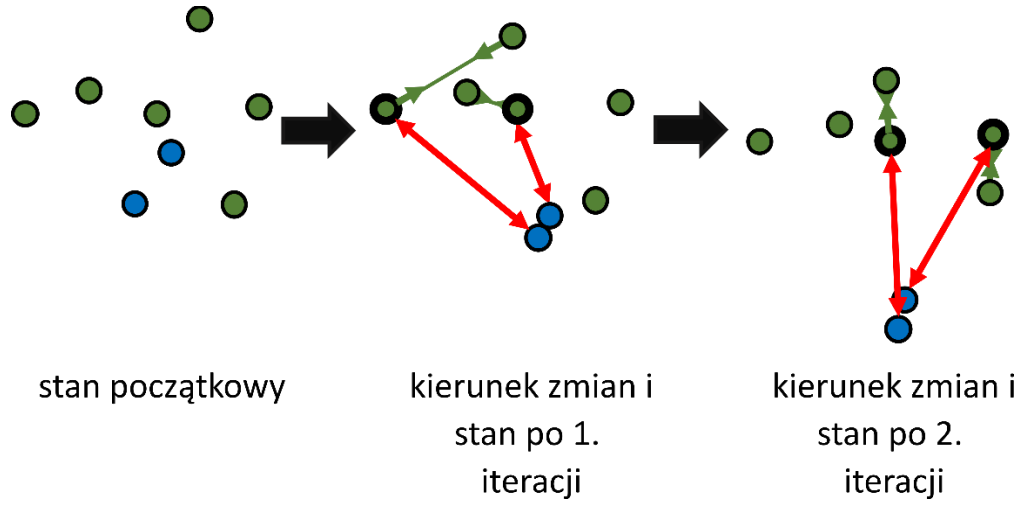
Drugim algorytmem badanym na laboratorium będzie algorytm straty trójkowej (ang. triplet loss), który jest w pewnym sensie modyfikacją wcześniej omawianego algorytmu straty kontrastowej. W przypadku algorytmu straty trójkowej wybierany jest najpierw punkt początkowy, tzw kotwica (ang. anchor), dla którego dobierany jest drugi punkt z tej samej klasy (tzw próbka pozytywna) oraz punkt z innej klasy (tzw próbka negatywna). Tak przygotowane trójki punktów wykorzystuje się w procesie treningu na podobnej zasadzie jak ma to miejsce w przypadku algorytmu straty kontrastowej, ale tym razem wzór funkcji straty ma postać następującą:

$$L_{\text{triplet}} = \max\left(0, m + d\left(r_a, r_p\right) - d(r_a, r_n)\right), \tag{2}$$

gdzie:

- L_{triplet} to wartość funkcji straty trójkowej,
- $d(r_a, r_p)$ to dystans pomiędzy reprezentacją kotwicy, a reprezentacją pozytywną (czyli punktami z tej samej klasy,
- $d(r_a, r_n)$ jest dystansem pomiędzy reprezentacją kotwicy, a reprezentacją próbki negatywnej (czyli pomiędzy punktami z różnych klas),
- m jest docelowym marginesem, który powinien oddzielać od siebie przykłady z różnych klas w przestrzeni reprezentacji.

Efekt wykonania algorytmu z według przedstawionych zasad skutkuje przybliżeniem do punktu kotwicy próbki pozytywnej i oddaleniem próbki negatywnej. Sytuację tę przedstawiono na rys. 5.



Rysunek 5. Przykład działania algorytmu wykorzystującego stratę trójkową. Punkty należą do dwóch klas (niebieskiej i zielonej) Pogrubionym konturem przedstawione są punkty wybrane jako tzw. Kotwice. Przybliżanie punktów pozytywnych zaznaczone jest zielonymi strzałkami, a oddalanie przykładów negatywnych – czerwonymi.

Pewną trudnością techniczną w algorytmie straty trójkowej jest pozyskiwanie trójek punktów, dla których wartość straty jest większa od zera. Proces ten zwiększa z iteracji na iterację czas poszukiwania tego typu trójek, gdyż już po kilku iteracjach algorytmu zdarzają się trójki punktów, które charakteryzują się zerową stratą (co oznacza, że w procesie treningu nie generują one poprawek do wag sieci). Z tego powodu z biegiem czasu algorytm straty trójkowej zwalnia, gdyż trójki przykładów treningowych stają się coraz trudniejsze do wyszukania.

4. Zadania do wykonania

Uwaga ogólna – każdy skrypt jest napisany w taki sposób, że po każdej epoce wagi sieci są zapisywane do pliku w formacie hdf. Kolejne wykonanie skryptu wznawia trening na zapisanych wagach, jeśli sieć ma być trenowana od zera (wag zainicjalizowanych losowo), to należy usunąć plik z wagami z folderu *_weights.*

W trakcie laboratorium do wykonania są następujące ćwiczenia:

1. Wytrenowanie algorytmu contrastive loss i zbadanie **wpływu jego nastaw, architektury, wybranego typu metryki dystansu** na proces treningu. W trakcie tego punktu zalecane jest uruchomienie **kilku próbnych krótkich treningów** algorytmu, by zbadać wpływ nastaw, a następnie wytrenować algorytm w dłuższym podejściu tak, aby uzyskać przestrzeń reprezentacji zapewaniającą separację między przynajmniej większością cyfr zbioru MNIST.
    a. Trening należy udokumentować **zamieszczając w sprawozdaniu widok przestrzeni decyzyjnej po wytrenowaniu** sieci.
    b. Podobnie należy opisać wpływ **poszczególnych hiperparametrów i innych czynników** na proces treningu.
2. Powtórzenie punktu pierwszego dla algorytmu triplet loss – badanie nastaw należy przeprowadzić na kilku próbnych treningach algorytmu, a następnie można przystąpić do

właściwego treningu sieci. Zalecane jest posługiwanie się względnie prostymi sieciami, gdyż trening za pomocą straty trójkowej ma tendencję do zwalniania wraz z postępem procesu treningu.

   a. Trening należy **udokumentować zamieszczając w sprawozdaniu widok przestrzeni decyzyjnej** po wytrenowaniu sieci.
   b. Podobnie należy **opisać wpływ poszczególnych hiperparametrów i innych czynników** na proces treningu.

3. Za pomocą modeli z poprzednich punktów należy **sparametryzować zbiór MNIST. Przez parametryzację rozumiemy tutaj wykorzystanie sieci neuronowej do wygenerowania reprezentacji obrazów cyfr ze zbioru MNIST – wygenerowane reprezentacje posłużą nam za parametry (cechy) wyliczone z obrazów ze zbioru.** Po parametryzacji zbiór powinien składać się z reprezentacji cyfr i oryginalnych etykiet ze zbioru. Następnie należy przygotować klasyfikator **k najbliższych sąsiadów** i zbadać jego **skuteczność działania** na reprezentacjach wygenerowanych przez algorytm **straty kontrastowej i trójkowej** (czyli **dla obu sieci**, najlepiej poprzez obliczenie **macierzy pomyłek**). Dla porównania **warto też sprawdzić skuteczność klasyfikatora działającego z siecią neuronową, której nie poddano jeszcze procesowi treningu (jedynie z zainicjalizowanymi losowo wagami, co zwykle dzieje się automatycznie przy utworzeniu nowej sieci)**.

   a. **Jakie wyniki** osiąga klasyfikator bazujący na reprezentacjach wygenerowanych przez algorytm, który **nie był trenowany**? Czy są one wyższe niż spodziewany efekt i z czego taka skuteczność może wynikać?
   b. **Przy wykorzystaniu których reprezentacji** (strata kontrastowa czy trójkowa) klasyfikator miał **wyższą skuteczność**? Z jakiego faktu może to wynikać?

   c. Czy klasyfikator popełniał jakieś **charakterystyczne pomyłki** i czy można je **powiązać jakoś z tym jak skonstruowana jest przestrzeń reprezentacji** wygenerowana przez algorytmy uczące się metryk odległości?

4. **Wybierz jeden z modeli** (albo wygenerowany przez algorytm straty kontrastowej, albo trójkowej). Zaprogramuj z jego pomocą **funkcję**, która przyjmuje jeden przykład ze zbioru MNIST, parametryzuje go za pomocą wybranego modelu i następnie **wyszukuje w zbiorze**
   a. **4 obrazki** obrazek najbardziej do niego **podobne**,
   b. **4 obrazki** najbardziej względem niego **różne**.

   Zamieść w sprawozdaniu kilka przykładów działania takiego prostego algorytmu wyszukiwania obrazka cyfry ze zbioru MNIST poprzez przykład.

5. Forma i zawartość sprawozdania

Ocenie podlegać będzie realizacja poszczególnych zadań w ramach danego ćwiczenia laboratoryjnego. Szczególnie premiowane będą dodatkowe wnioski i obserwacje poczynione samodzielnie przez osoby wykonujące ćwiczenie. Zarówno te dotyczące różnego rodzaju teoretycznych aspektów wykonywanych czynności, jak i ich możliwych praktycznych zastosowań. Poszczególnym zadaniom przydzielono liczbę punktów jakie można zdobyć za ich wykonania. Maksymalna liczba punktów wynosi 10.

Dodatki:

Pliki instruktażowe oraz pliki z danymi, niezbędne do wykonania ćwiczenia laboratoryjnego:

1) *contrastive loss.ipynb*
2) *triplet loss.ipynb*